

Chapter 5 Exam Notes: Input/Output

Modern Operating Systems Study Notes

Contents

1	Scope	2
2	5.1 Principles of I/O Hardware	2
2.1	I/O Devices	2
2.2	Device Controller	2
2.3	Memory-Mapped I/O	2
2.4	Port-Mapped I/O	2
2.5	DMA	2
3	Interrupts: How They Occur and Are Handled	3
3.1	How Interrupt Occurs	3
3.2	Interrupt Handling Steps	3
3.3	Precise vs Imprecise Interrupt	3
4	5.2 Principles of I/O Software	3
4.1	Goals	3
4.2	Programmed I/O	4
4.3	Interrupt-Driven I/O	4
4.4	DMA-Based I/O	4
5	5.3 I/O Software Layers	4
5.1	Layer Responsibility Examples	4
6	Spooling	4
7	Math-Based Problems	5
7.1	DMA Bus Bandwidth	5

7.2	Interrupt Rate	5
7.3	Printer Interrupt CPU Cost	5
7.4	Disk RPM	5
7.5	Sector Time	5
7.6	Disk Data Rate	5
7.7	Cylinder Skew	5
7.8	RAID Failure Probability	6
7.9	Clock Register	6
7.10	Display Memory	6
7.11	Thin Client Bandwidth	6
7.12	Power Saving	6
8	Likely Questions with Answers	6

1 Scope

Exam focus: Study I/O concepts in 5.1, 5.2, and 5.3. Know how interrupts occur and are handled. Skip from 5.4 onward for large theory, but math-based problems from disks/clocks/display may still appear, so formulas are included.

2 5.1 Principles of I/O Hardware

2.1 I/O Devices

- **Block devices:** store data in addressable blocks, e.g., disks/SSDs.
- **Character devices:** stream characters/bytes, e.g., keyboard, mouse, serial port.

2.2 Device Controller

A controller is hardware that manages a device and exposes registers to the CPU. The OS usually talks to the controller, not directly to the mechanical/electrical device.

2.3 Memory-Mapped I/O

Device registers are mapped into the physical address space. CPU reads/writes those addresses to control devices.

2.4 Port-Mapped I/O

Device registers live in a separate I/O address space and require special I/O instructions.

2.5 DMA

Direct Memory Access lets a controller transfer blocks between device and memory without CPU copying each word.

DMA flow:

1. CPU programs DMA controller with source, destination, count, and direction.
2. DMA controller requests bus.
3. Data is transferred.
4. Controller interrupts CPU when done.

3 Interrupts: How They Occur and Are Handled

3.1 How Interrupt Occurs

1. Device finishes work or needs attention.
2. Controller asserts interrupt signal.
3. CPU finishes current safe point.
4. CPU switches to kernel mode and jumps to interrupt handler.

3.2 Interrupt Handling Steps

1. Save CPU context.
2. Identify interrupt source.
3. Acknowledge interrupt controller/device.
4. Run device-specific handler.
5. Move data/status to OS structures.
6. Wake blocked process if needed.
7. Restore context or schedule another process.

3.3 Precise vs Imprecise Interrupt

- **Precise interrupt:** machine state clearly shows all instructions before PC completed and none after PC completed.
- **Imprecise interrupt:** hard to know exactly which instructions finished, common issue in highly pipelined/out-of-order CPUs.

4 5.2 Principles of I/O Software

4.1 Goals

- Device independence.
- Uniform naming.
- Error handling close to the source.
- Synchronous and asynchronous I/O support.
- Buffering.
- Sharing vs dedicated devices.

4.2 Programmed I/O

CPU repeatedly checks device status and transfers data itself.

- Simple.
- Wastes CPU while waiting.

4.3 Interrupt-Driven I/O

CPU starts I/O and does other work. Device interrupts when ready/done.

- Better CPU use.
- Interrupt overhead for frequent events.

4.4 DMA-Based I/O

Best for bulk transfer. CPU overhead is mostly setup and completion handling.

5 5.3 I/O Software Layers

1. **Interrupt handlers:** lowest software response to hardware interrupt.
2. **Device drivers:** device-specific control and command logic.
3. **Device-independent I/O software:** naming, buffering, allocation, protection, error reporting.
4. **User-space I/O software:** libraries, spooling daemons, formatting tools.

5.1 Layer Responsibility Examples

- Compute disk track/sector/head: device driver.
- Write command to controller register: device driver.
- Check permission to use a device: device-independent software.
- Convert integer to printable ASCII: user-level library.

6 Spooling

Spooling uses disk as a queue for dedicated devices such as printers. Users submit jobs quickly; a daemon prints them one at a time.

7 Math-Based Problems

7.1 DMA Bus Bandwidth

If one transaction takes T seconds and transfers B bytes:

$$\text{bandwidth} = \frac{B}{T}$$

Example: 4 bytes per 100 nsec gives $4/(100 \times 10^{-9}) = 40 \text{ MB/sec}$.

7.2 Interrupt Rate

If saving/restoring uses N memory words and each access costs t :

$$\begin{aligned} \text{overhead} &= Nt \\ \text{max interrupts/sec} &\approx \frac{1}{\text{overhead}} \end{aligned}$$

7.3 Printer Interrupt CPU Cost

$$\text{CPU fraction} = \frac{\text{chars/sec} \times \text{interrupt cost}}{1 \text{ sec}}$$

7.4 Disk RPM

$$\begin{aligned} \text{rotations/sec} &= \frac{\text{RPM}}{60} \\ \text{rotation time} &= \frac{1}{\text{rotations/sec}} \end{aligned}$$

7.5 Sector Time

$$\text{sector time} = \frac{\text{rotation time}}{\text{sectors per track}}$$

7.6 Disk Data Rate

$$\text{bytes/sec} = \text{sectors/rotation} \times \text{bytes/sector} \times \text{rotations/sec}$$

7.7 Cylinder Skew

If track-to-track seek takes s msec and rotation time is r msec:

$$\text{skew sectors} = \frac{s}{r} \times \text{sectors per track}$$

7.8 RAID Failure Probability

For k drives, each failing in an hour with probability p , RAID fails if at least two fail:

$$P(\text{fail}) = 1 - (1 - p)^k - kp(1 - p)^{k-1}$$

For small p :

$$P(\text{fail}) \approx \binom{k}{2} p^2$$

7.9 Clock Register

For crystal frequency f and desired tick interval T :

$$\text{holding register} = fT$$

Example: 500 MHz and 1 msec gives $500 \times 10^6 \times 10^{-3} = 500000$.

7.10 Display Memory

$$\text{bytes} = \text{width} \times \text{height} \times \text{bytes per pixel}$$

For 24-bit color, bytes per pixel = 3.

7.11 Thin Client Bandwidth

$$\text{bits/sec} = \text{width} \times \text{height} \times \text{bytes/pixel} \times 8 \times \text{frames/sec}$$

7.12 Power Saving

If voltage is reduced by factor n :

$$\text{power} = \frac{1}{n^2}, \quad \text{speed} = \frac{1}{n}$$

Choose largest n that still finishes required work before the deadline.

8 Likely Questions with Answers

Question

Q1. Block vs character devices

Answer

Block devices store information in fixed-size addressable blocks and allow random access, such as disks and SSDs. Character devices deliver or accept a stream of bytes without block addressing, such as keyboards, mice, and serial ports.

Question

Q2. Memory-mapped vs port-mapped I/O

Answer

In memory-mapped I/O, device registers appear as memory addresses, so normal load/store instructions access them. In port-mapped I/O, device registers are in a separate I/O address space and special I/O instructions are used. Memory-mapped I/O simplifies programming but must be protected from ordinary user access.

Question

Q3. DMA steps and benefits

Answer

The CPU programs the DMA controller with source, destination, count, and direction. The DMA controller transfers data between device and memory, then interrupts the CPU when done. DMA reduces CPU overhead because the CPU does not copy every byte or word itself.

Question

Q4. How does an interrupt occur and how is it handled?

Answer

A device controller raises an interrupt when it completes work or needs attention. The CPU switches to kernel mode and jumps to the interrupt handler. The handler saves context, identifies and acknowledges the interrupt, performs device-specific work, wakes blocked processes if needed, restores/schedules execution, and returns from interrupt.

Question

Q5. Programmed I/O vs interrupt-driven I/O vs DMA

Answer

Programmed I/O makes the CPU poll the device and copy data, which is simple but wastes CPU time. Interrupt-driven I/O lets the CPU do other work while the device operates, but each interrupt has overhead. DMA transfers blocks directly between device and memory, giving the lowest CPU overhead for large transfers.

Question

Q6. Four I/O software layers and responsibilities

Answer

Interrupt handlers perform urgent device completion work. Device drivers issue device-specific commands and manage controller registers. Device-independent I/O software provides naming, buffering, caching, protection, and uniform interfaces. User-level I/O software includes libraries, daemons, and spooling systems used by applications.

Question

Q7. How are common I/O math problems solved?

Answer

Use units carefully. DMA bandwidth is bytes per transfer $\times$ transfers per second. Interrupt overhead is interrupt rate $\times$ time per interrupt. Average rotational delay is half a rotation: $60/(2 \times RPM)$ seconds. Sector time is rotation time divided by sectors per track. Display bandwidth is pixels $\times$ bytes per pixel $\times$ frames per second. For RAID or reliability probability, identify independent failure probabilities and multiply only when independence is stated.